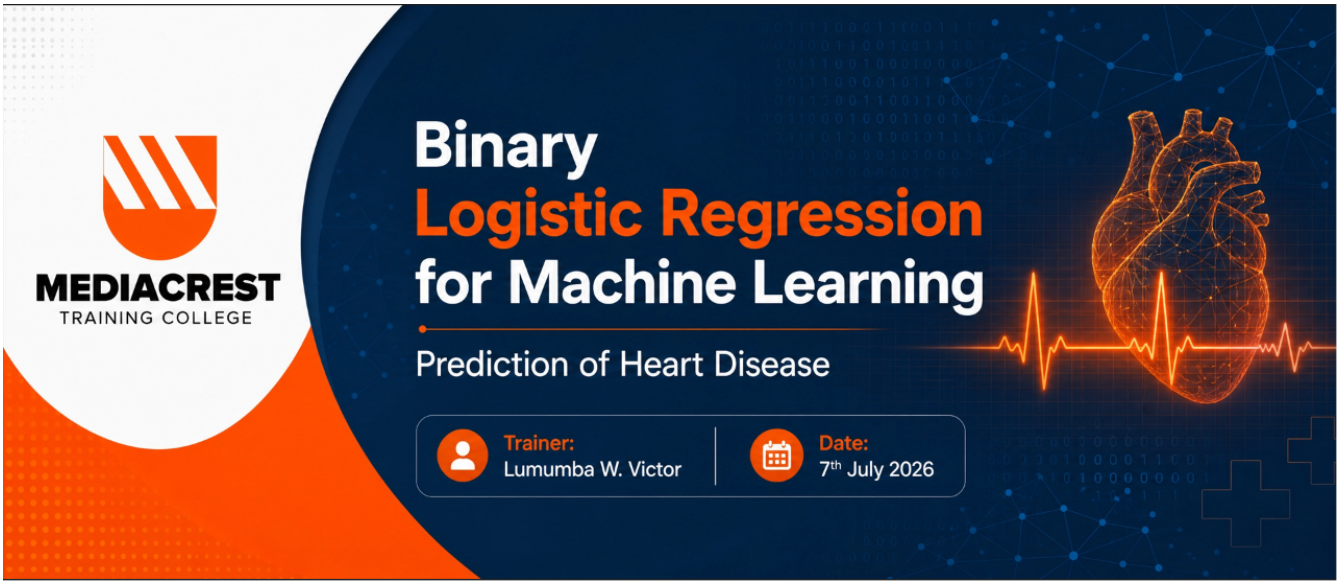


1. LOAD THE NEEDED LIBRARIES

In [1]:

```
### Load the libraries for data importation, numeric manipulation and visualization
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm

### Load the libraries for data preprocessing and data partitioning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc

import warnings
warnings.filterwarnings("ignore")
```

2. LOAD THE DATASET

In [2]:

```
df = pd.read_csv('Heart.csv', na_values=['NA', '?', ''])
df.head()
```

Out[2]:

	Age	Sex	ChestPain	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	T
0	63	1	typical	145	233	1	2	150	0	2.3	3	0.0	f
1	67	1	asymptomatic	160	286	0	2	108	1	1.5	2	3.0	nor
2	67	1	asymptomatic	120	229	0	2	129	1	2.6	2	2.0	revers
3	37	1	nonanginal	130	250	0	0	187	0	3.5	3	0.0	nor
4	41	0	nontypical	130	204	0	2	172	0	1.4	1	0.0	nor

Drop AHD Since it is a String Version of the Target Variable HD

In [3]:

```
df = df.drop(columns = ['AHD'])
df.head()
```

Out[3]:

	Age	Sex	ChestPain	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	T
0	63	1	typical	145	233	1	2	150	0	2.3	3	0.0	f
1	67	1	asymptomatic	160	286	0	2	108	1	1.5	2	3.0	nor
2	67	1	asymptomatic	120	229	0	2	129	1	2.6	2	2.0	revers
3	37	1	nonanginal	130	250	0	0	187	0	3.5	3	0.0	nor
4	41	0	nontypical	130	204	0	2	172	0	1.4	1	0.0	nor

Handling Missing Values

In [4]:

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Age        303 non-null    int64  
 1   Sex        303 non-null    int64  
 2   ChestPain  303 non-null    str    
 3   RestBP     303 non-null    int64  
 4   Chol       303 non-null    int64  
 5   Fbs        303 non-null    int64  
 6   RestECG    303 non-null    int64  
 7   MaxHR      303 non-null    int64  
 8   ExAng      303 non-null    int64  
 9   Oldpeak    303 non-null    float64
 10  Slope      303 non-null    int64  
 11  Ca         299 non-null    float64
 12  Thal       301 non-null    str    
 13  HD         303 non-null    int64  
dtypes: float64(2), int64(10), str(2)
memory usage: 38.7 KB
```

Check the Sum of Missing Values per Variable

In [5]:

```
df.isna().sum()
```

Out[5]:

```
Age          0
Sex          0
ChestPain    0
RestBP       0
Chol         0
```

```
Fbs          0
RestECG      0
MaxHR        0
ExAng        0
Oldpeak      0
Slope        0
Ca           4
Thal         2
HD           0
dtype: int64
```

Data Imputation (Median and Mode)

In [6]:

```
df['Ca'] = df['Ca'].fillna(df['Ca'].median())
df['Thal'] = df['Thal'].fillna(df['Thal'].mode()[0])
```

In [7]:

```
df.isna().sum()
```

Out[7]:

```
Age          0
Sex          0
ChestPain    0
RestBP       0
Chol         0
Fbs          0
RestECG      0
MaxHR        0
ExAng        0
Oldpeak      0
Slope        0
Ca           0
Thal         0
HD           0
dtype: int64
```

In [8]:

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Age        303 non-null    int64
 1   Sex        303 non-null    int64
 2   ChestPain  303 non-null    str
 3   RestBP     303 non-null    int64
 4   Chol       303 non-null    int64
 5   Fbs        303 non-null    int64
 6   RestECG    303 non-null    int64
 7   MaxHR      303 non-null    int64
 8   ExAng      303 non-null    int64
 9   Oldpeak    303 non-null    float64
 10  Slope      303 non-null    int64
 11  Ca         303 non-null    float64
 12  Thal       303 non-null    str
 13  HD         303 non-null    int64
```

```
dtypes: float64(2), int64(10), str(2)
memory usage: 38.7 KB
```

Seperate X-Matrix of Predictor and y-Predictor Variable

In [9]:

```
X = df.drop(columns = ['HD'])
X.head()
```

Out[9]:

	Age	Sex	ChestPain	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	T
0	63	1	typical	145	233	1	2	150	0	2.3	3	0.0	f
1	67	1	asymptomatic	160	286	0	2	108	1	1.5	2	3.0	nor
2	67	1	asymptomatic	120	229	0	2	129	1	2.6	2	2.0	revers
3	37	1	nonanginal	130	250	0	0	187	0	3.5	3	0.0	nor
4	41	0	nontypical	130	204	0	2	172	0	1.4	1	0.0	nor

In [10]:

```
y = df['HD']
y.head()
```

Out[10]:

```
0    0
1    1
2    1
3    0
4    0
Name: HD, dtype: int64
```

Get Dummies for Character Variables

In [11]:

```
df['ChestPain'].value_counts()
```

Out[11]:

```
ChestPain
asymptomatic    144
nonanginal       86
nontypical       50
typical          23
Name: count, dtype: int64
```

In [12]:

```
df['Thal'].value_counts()
```

Out[12]:

```
Thal
normal        168
reversable    117
fixed          18
Name: count, dtype: int64
```

In [13]:

```
X = pd.get_dummies(X, drop_first = True)
X.head()
```

Out[13]:

	Age	Sex	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	ChestPain_nonangin
0	63	1	145	233	1	2	150	0	2.3	3	0.0	Fals
1	67	1	160	286	0	2	108	1	1.5	2	3.0	Fals
2	67	1	120	229	0	2	129	1	2.6	2	2.0	Fals
3	37	1	130	250	0	0	187	0	3.5	3	0.0	Tru
4	41	0	130	204	0	2	172	0	1.4	1	0.0	Fals

Convert the Boolean Data Types to Binary

In [14]:

```
bool_cols = X.select_dtypes(include = 'bool').columns
X[bool_cols] = X[bool_cols].astype(int)
X.head()
```

Out[14]:

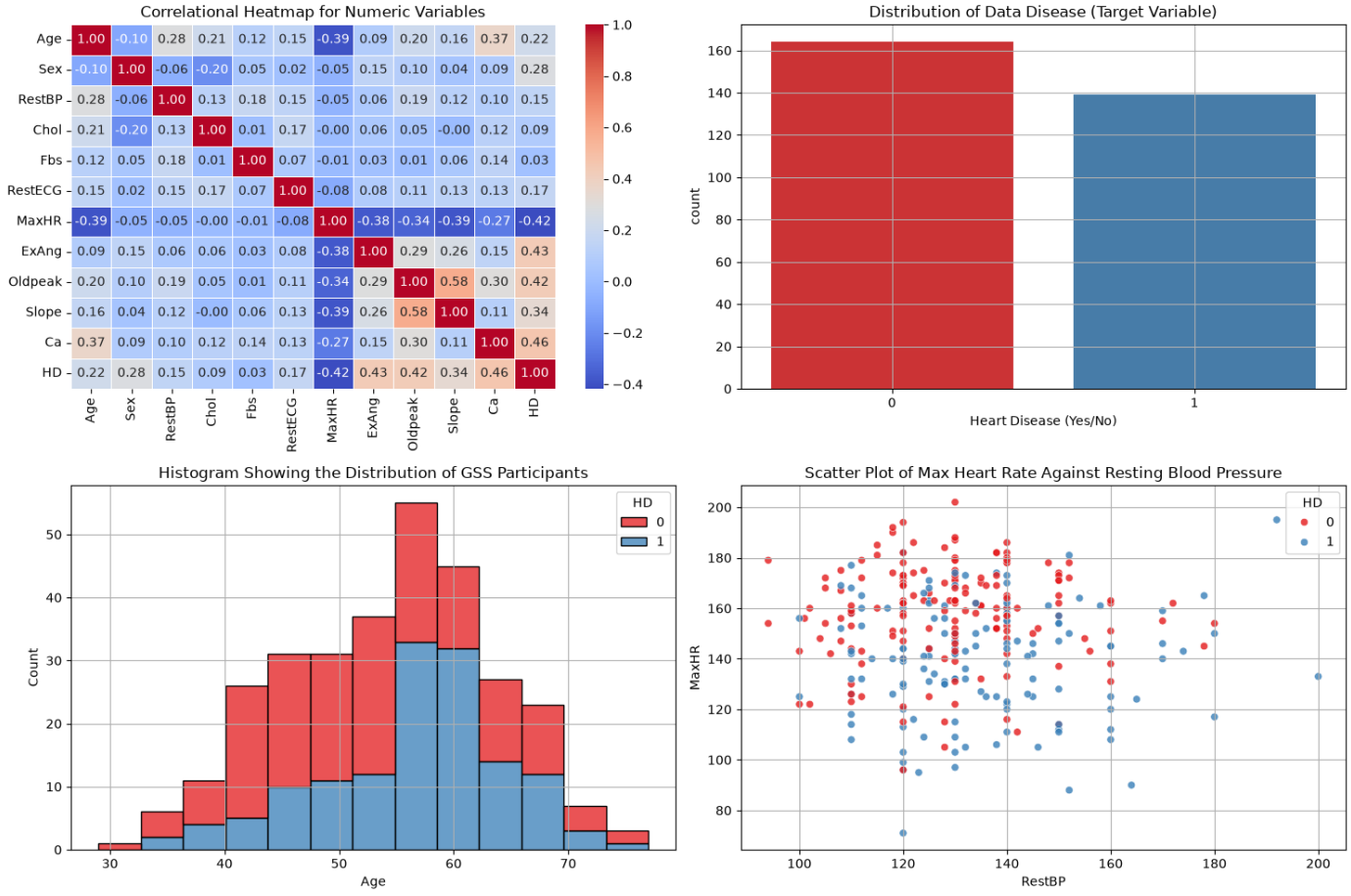
	Age	Sex	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	ChestPain_nonangin
0	63	1	145	233	1	2	150	0	2.3	3	0.0	
1	67	1	160	286	0	2	108	1	1.5	2	3.0	
2	67	1	120	229	0	2	129	1	2.6	2	2.0	
3	37	1	130	250	0	0	187	0	3.5	3	0.0	
4	41	0	130	204	0	2	172	0	1.4	1	0.0	

3. EXPLORATORY DATA ANALYSIS (EDA)

In [29]:

```
plt.figure(figsize = (15, 10))

# 1. Correlational Heatmap
plt.subplot(2,2, 1)
numeric_df = df.select_dtypes(include = [np.number])
sns.heatmap(numeric_df.corr(), annot = True, cmap = 'coolwarm', fmt=".2f", linewidths =
plt.title("Correlational Heatmap for Numeric Variables")

# 2. Distribution of the Target Variables
plt.subplot(2,2,2)
sns.countplot(x = "HD", data = df, palette = "Set1")
plt.title("Distribution of Data Disease (Target Variable)")
plt.xlabel("Heart Disease (Yes/No)")
plt.grid(True)

# 3. Age Distribution of the Respondent in the GSS
plt.subplot(2,2,3)
sns.histplot(data = df, x = "Age",
```

```
            hue = "HD",
            multiple = "stack",
            palette = "Set1")
plt.title("Histogram Showing the Distribution of GSS Participants")
plt.grid(True)

# 4. Scatter Plot of Maximum Heart Rate and Resting Blood Pressure
plt.subplot(2,2,4)
sns.scatterplot(x = "RestBP", y = "MaxHR",
                data = df,
                hue = "HD",
                palette = "Set1",
                alpha = 0.8)
plt.title("Scatter Plot of Max Heart Rate Against Resting Blood Pressure")
plt.grid(True)

plt.tight_layout()
plt.show()
```



4. DATA PARTITIONING

In [45]:

```
# Split the Data into 80% Training Set and 20% Testing Set
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size = 0.2,
                                                    random_state = 42,
                                                    stratify = y)
```

In [44]:

```
X_train.head()
```

Out[44]:

	Age	Sex	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	ChestPain_nonang
180	48	1	124	274	0	2	166	0	0.5	2	0.0	
208	55	1	130	262	0	0	155	0	0.0	1	0.0	
167	54	0	132	288	1	2	159	1	0.0	1	1.0	
105	54	1	108	309	0	0	156	0	0.0	1	0.0	
297	57	0	140	241	0	0	123	1	0.2	2	0.0	

5. STATISTICAL APPROACH TO BINARY LOGISTIC REGRESSION ANALYSIS

5.1. Add Constant to the X_train Data Set

In [46]:

```
X_train = sm.add_constant(X_train)
```

In [47]:

```
X_train.head()
```

Out[47]:

	const	Age	Sex	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	ChestPain_
180	1.0	48	1	124	274	0	2	166	0	0.5	2	0.0	
208	1.0	55	1	130	262	0	0	155	0	0.0	1	0.0	
167	1.0	54	0	132	288	1	2	159	1	0.0	1	1.0	
105	1.0	54	1	108	309	0	0	156	0	0.0	1	0.0	
297	1.0	57	0	140	241	0	0	123	1	0.2	2	0.0	

In [49]:

```
X_test.head()
```

Out[49]:

	Age	Sex	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	ChestPain_nonang
219	59	1	138	271	0	2	182	0	0.0	1	0.0	
271	66	1	160	228	0	2	138	0	2.3	1	0.0	
89	51	0	130	256	0	2	149	0	0.5	1	0.0	
101	34	1	118	182	0	2	174	0	0.0	1	0.0	
67	54	1	150	232	0	2	165	0	1.6	1	0.0	

In [48]:

```
y_train.head()
```

Out[48]:

```
180    1
208    0
```

```
167    0
105    0
297    1
Name: HD, dtype: int64
```

In [50]:

```
y_test.head()
```

Out[50]:

```
219    0
271    0
89     0
101    0
67     0
Name: HD, dtype: int64
```

5.2. Fit the Binary Logistic Regression Model

In [54]:

```
model = sm.Logit(y_train, X_train).fit(disp = 0)
# when disp = 0, We surppress the optimization  output
```

5.3. Print the Results of the Model

In [56]:

```
print(model.summary())
```

```
                           Logit Regression Results
==============================================================================
Dep. Variable:                     HD   No. Observations:                  242
Model:                          Logit   Df Residuals:                      225
Method:                           MLE   Df Model:                           16
Date:                Wed, 08 Jul 2026   Pseudo R-squ.:                  0.5013
Time:                        19:58:13   Log-Likelihood:                -83.237
converged:                       True   LL-Null:                       -166.91
Covariance Type:            nonrobust   LLR p-value:                 2.835e-27
==============================================================================
                             coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const                     -3.9477      3.143     -1.256      0.209     -10.108       2.212
Age                       -0.0110      0.028     -0.401      0.689      -0.065       0.043
Sex                        1.5864      0.553      2.871      0.004       0.503       2.669
RestBP                     0.0206      0.012      1.653      0.098      -0.004       0.045
Chol                       0.0050      0.004      1.205      0.228      -0.003       0.013
Fbs                       -0.3198      0.621     -0.515      0.607      -1.538       0.898
RestECG                    0.2211      0.207      1.069      0.285      -0.184       0.627
MaxHR                     -0.0180      0.012     -1.512      0.130      -0.041       0.005
ExAng                      0.6743      0.479      1.409      0.159      -0.264       1.612
Oldpeak                    0.1830      0.281      0.651      0.515      -0.368       0.734
Slope                      0.7391      0.425      1.740      0.082      -0.093       1.572
Ca                         1.3944      0.321      4.340      0.000       0.765       2.024
ChestPain_nonanginal      -1.6332      0.542     -3.013      0.003      -2.696      -0.571
ChestPain_nontypical      -0.9614      0.603     -1.595      0.111      -2.142       0.220
ChestPain_typical         -2.0153      0.714     -2.821      0.005      -3.415      -0.615
Thal_normal               -0.2242      0.921     -0.243      0.808      -2.030       1.581
Thal_reversable            1.2808      0.911      1.406      0.160      -0.505       3.067
==============================================================================
```

5.4. Odds Ratio

In [59]:

```
params = model.params
conf = model.conf_int()
conf['OR'] = params
conf.colums = ['2.5%','97.5%', 'Odds Ratio']
print(np.exp(conf))
```

```
                              0          1        OR
const                  0.000041   9.135949  0.019298
Age                    0.936986   1.043937  0.989017
Sex                    1.654193  14.432281  4.886080
RestBP                 0.996183   1.045995  1.020785
Chol                   0.996904   1.013088  1.004964
Fbs                    0.214893   2.454609  0.726278
RestECG                0.831611   1.871197  1.247441
MaxHR                  0.959512   1.005341  0.982159
ExAng                  0.768092   5.015130  1.962671
Oldpeak                0.692060   2.083557  1.200811
Slope                  0.910950   4.814183  2.094154
Ca                     2.148307   7.569922  4.032682
ChestPain_nonanginal   0.067505   0.565071  0.195308
ChestPain_nontypical   0.117370   1.245659  0.382364
ChestPain_typical      0.032861   0.540549  0.133277
Thal_normal            0.131377   4.861040  0.799144
Thal_reversable        0.603402  21.471568  3.599442
```

6. MACHINE LEARNING APPROACH FOR BINARY LOGISTIC REGRESSION

In [60]:

```
X_train.head()
```

Out[60]:

	const	Age	Sex	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	ChestPain_
180	1.0	48	1	124	274	0	2	166	0	0.5	2	0.0	
208	1.0	55	1	130	262	0	0	155	0	0.0	1	0.0	
167	1.0	54	0	132	288	1	2	159	1	0.0	1	1.0	
105	1.0	54	1	108	309	0	0	156	0	0.0	1	0.0	
297	1.0	57	0	140	241	0	0	123	1	0.2	2	0.0	

6.1. Split the Data into 80% Training Set and 20% Testing Set

In [64]:

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size = 0.2,
                                                    random_state = 42,
                                                    stratify = y)
```

In [65]:

```
X_train.head()
```

Out[65]:

	Age	Sex	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	ChestPain_nonang
180	48	1	124	274	0	2	166	0	0.5	2	0.0	
208	55	1	130	262	0	0	155	0	0.0	1	0.0	
167	54	0	132	288	1	2	159	1	0.0	1	1.0	
105	54	1	108	309	0	0	156	0	0.0	1	0.0	
297	57	0	140	241	0	0	123	1	0.2	2	0.0	

6.2. Features Scaling

In [66]:

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
```

In [68]:

```
X_train
```

Out[68]:

```
array([[-0.72948484,  0.68313005, -0.39569185, ..., -0.29189346,
        -1.13270423,  1.24393264],
       [ 0.05016647,  0.68313005, -0.05451337, ..., -0.29189346,
         0.882843  , -0.80390205],
       [-0.06121229, -1.46385011,  0.0592128 , ..., -0.29189346,
         0.882843  , -0.80390205],
       ...,
       [ 1.38671155,  0.68313005, -0.62314418, ..., -0.29189346,
        -1.13270423,  1.24393264],
       [ 1.60946907,  0.68313005,  0.51411744, ..., -0.29189346,
        -1.13270423,  1.24393264],
       [-0.72948484,  0.68313005, -1.19177499, ..., -0.29189346,
        -1.13270423,  1.24393264]], shape=(242, 16))
```

In [69]:

```
X_test
```

Out[69]:

```
array([[ 0.54416588,  0.6984303 ,  0.19817758,  0.8137621 , -0.44280744,
         0.99247309,  1.43523731, -0.6984303 , -0.93138063, -1.05045146,
        -0.81371085, -0.75106762, -0.38851434, -0.26490647,  0.95197164,
        -0.75106762],
       [ 1.30599811,  0.6984303 ,  1.4768041 , -0.13744021, -0.44280744,
         0.99247309, -0.43293773, -0.6984303 ,  0.85376558, -1.05045146,
        -0.81371085, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
        -0.75106762],
       [-0.32649953, -1.43178211, -0.26677751,  0.48194734, -0.44280744,
         0.99247309,  0.03410603, -0.6984303 , -0.54330537, -1.05045146,
        -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
        -0.75106762],
       [-2.17666352,  0.6984303 , -0.96421016, -1.15500547, -0.44280744,
         0.99247309,  1.09556912, -0.6984303 , -0.93138063, -1.05045146,
        -0.81371085, -0.75106762, -0.38851434,  3.77491722,  0.95197164,
```

```
 -0.75106762],
[ 0.        ,  0.6984303 ,  0.89561023, -0.04895628, -0.44280744,
  0.99247309,  0.71344241, -0.6984303 ,  0.31046021, -1.05045146,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[ 0.65299906, -1.43178211, -0.84797138, -1.24348941,  2.25831796,
 -1.0597594 , -2.21619573, -0.6984303 , -0.93138063, -1.05045146,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.97949858, -1.43178211,  0.31441636, -0.86743268, -0.44280744,
 -1.0597594 ,  1.30786174, -0.6984303 , -0.93138063, -1.05045146,
  1.02467292, -0.75106762,  2.57390754, -0.26490647,  0.95197164,
 -0.75106762],
[ 2.17666352, -1.43178211, -0.84797138,  0.76952013, -0.44280744,
  0.99247309, -1.15473263,  1.43178211, -0.77615053, -1.05045146,
  0.10548104, -0.75106762,  2.57390754, -0.26490647,  0.95197164,
 -0.75106762],
[-2.06783034,  0.6984303 , -0.49925506,  1.05709292, -0.44280744,
  0.99247309,  0.33131569,  1.43178211, -0.93138063, -1.05045146,
 -0.81371085, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[-0.97949858, -1.43178211,  0.19817758,  0.03952766, -0.44280744,
  0.99247309,  0.1614816 ,  1.43178211, -0.77615053,  0.55148702,
 -0.81371085, -0.75106762, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.87066541,  0.6984303 , -0.84797138,  1.03497194, -0.44280744,
  0.99247309, -1.91898606, -0.6984303 ,  0.15523011,  0.55148702,
  0.10548104, -0.75106762,  2.57390754, -0.26490647, -1.05045146,
  1.33143805],
[ 0.4353327 , -1.43178211,  0.89561023,  1.0792139 ,  2.25831796,
  0.99247309,  0.58606683, -0.6984303 , -0.15523011, -1.05045146,
 -0.81371085, -0.75106762, -0.38851434,  3.77491722,  0.95197164,
 -0.75106762],
[-1.74133082,  0.6984303 ,  0.19817758, -1.30985236, -0.44280744,
 -1.0597594 ,  1.05311059, -0.6984303 , -0.93138063, -1.05045146,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 1.30599811,  0.6984303 ,  1.4768041 ,  0.2607375 , -0.44280744,
 -1.0597594 , -1.19719116,  1.43178211, -0.93138063,  0.55148702,
  1.9438648 , -0.75106762,  2.57390754, -0.26490647, -1.05045146,
 -0.75106762],
[ 0.        ,  0.6984303 , -0.84797138, -1.02227957, -0.44280744,
 -1.0597594 , -1.49440082, -0.6984303 ,  0.15523011,  0.55148702,
  0.10548104, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[ 0.54416588,  0.6984303 ,  1.70928165, -1.28773138,  2.25831796,
  0.99247309, -2.47094687, -0.6984303 , -0.15523011,  0.55148702,
  1.02467292, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
 -0.75106762],
[ 0.4353327 ,  0.6984303 , -0.15053874, -0.22592415, -0.44280744,
  0.99247309,  1.05311059, -0.6984303 ,  1.55230105, -1.05045146,
  1.02467292,  1.33143805, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[ 1.63249764,  0.6984303 ,  1.4768041 , -0.00471431,  2.25831796,
  0.99247309, -0.7301474 , -0.6984303 , -0.85376558,  0.55148702,
  0.10548104, -0.75106762, -0.38851434,  3.77491722,  0.95197164,
 -0.75106762],
[ 0.87066541, -1.43178211,  0.31441636,  0.74739915, -0.44280744,
  0.99247309,  0.50114979, -0.6984303 ,  1.86276126,  2.1534255 ,
```

```
  1.02467292, -0.75106762, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.32649953,  0.6984303 , -0.26677751, -2.28317565, -0.44280744,
 -1.0597594 , -1.40948378,  1.43178211,  0.        ,  0.55148702,
  0.10548104, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[ 0.54416588,  0.6984303 ,  0.02381942, -0.00471431, -0.44280744,
 -1.0597594 ,  0.54360831, -0.6984303 , -0.54330537,  0.55148702,
 -0.81371085, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[-1.30599811,  0.6984303 , -0.26677751, -1.19924744, -0.44280744,
 -1.0597594 ,  0.07656455, -0.6984303 , -0.93138063, -1.05045146,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[-1.19716494, -1.43178211, -0.15053874,  2.36223097,  2.25831796,
  0.99247309, -0.51785478,  1.43178211,  1.39707095,  0.55148702,
 -0.81371085, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[-0.10883318,  0.6984303 , -0.26677751,  0.2607375 ,  2.25831796,
  0.99247309,  1.05311059, -0.6984303 , -0.93138063, -1.05045146,
  1.9438648 ,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.10883318, -1.43178211,  2.63919184,  2.0525372 , -0.44280744,
 -0.03364316, -1.32456673,  1.43178211,  1.70753116,  0.55148702,
 -0.81371085, -0.75106762, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[-1.63249764, -1.43178211,  0.19817758, -0.31440808, -0.44280744,
 -1.0597594 ,  0.1614816 , -0.6984303 , -0.93138063,  0.55148702,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.        , -1.43178211,  1.4768041 , -0.73470678, -0.44280744,
 -1.0597594 ,  0.62852536, -0.6984303 , -0.93138063, -1.05045146,
  0.10548104,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[-0.54416588,  0.6984303 , -0.84797138, -1.02227957, -0.44280744,
 -1.0597594 , -0.39047921, -0.6984303 ,  0.62092042,  0.55148702,
  1.9438648 ,  1.33143805, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[-1.85016399,  0.6984303 , -0.26677751,  0.34922143, -0.44280744,
 -1.0597594 ,  1.64752993, -0.6984303 ,  1.78514621,  2.1534255 ,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.76183223,  0.6984303 ,  0.19817758, -1.50894122, -0.44280744,
  0.99247309, -0.98489854,  1.43178211,  1.86276126,  0.55148702,
  0.10548104, -0.75106762, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[-1.41483129,  0.6984303 ,  0.02381942, -0.69046481, -0.44280744,
 -1.0597594 , -0.68768887, -0.6984303 , -0.93138063,  0.55148702,
 -0.81371085, -0.75106762,  2.57390754, -0.26490647, -1.05045146,
 -0.75106762],
[ 0.4353327 , -1.43178211,  2.05799797, -0.20380316,  2.25831796,
  0.99247309, -0.09326954,  1.43178211,  1.24184084,  0.55148702,
  1.02467292, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
 -0.75106762],
[-1.08833176,  0.6984303 , -0.84797138, -1.44257826, -0.44280744,
 -1.0597594 , -0.17818659,  1.43178211,  1.24184084,  2.1534255 ,
 -0.81371085, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
 -0.75106762],
[ 0.4353327 ,  0.6984303 , -0.55737445, -0.31440808, -0.44280744,
```

```
 -1.0597594 , -0.17818659, -0.6984303 , -0.62092042,  0.55148702,
 -0.81371085, -0.75106762,  2.57390754, -0.26490647, -1.05045146,
  1.33143805],
[-0.97949858,  0.6984303 , -1.13856832,  0.57043127, -0.44280744,
  0.99247309,  1.56261288, -0.6984303 , -0.93138063, -1.05045146,
 -0.81371085, -0.75106762, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.4353327 ,  0.6984303 , -0.55737445,  1.45527063, -0.44280744,
  0.99247309,  0.96819355, -0.6984303 , -0.93138063, -1.05045146,
  1.02467292, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[-1.41483129,  0.6984303 , -0.26677751, -0.44713399, -0.44280744,
  0.99247309,  0.84081798, -0.6984303 ,  0.62092042,  0.55148702,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.65299906,  0.6984303 ,  0.31441636, -1.08864252, -0.44280744,
  0.99247309,  0.28885717, -0.6984303 ,  1.39707095,  0.55148702,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[-0.21766635,  0.6984303 ,  0.19817758, -0.24804513, -0.44280744,
 -1.0597594 ,  0.8832765 , -0.6984303 , -0.93138063, -1.05045146,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.76183223,  0.6984303 ,  0.77937145, -0.69046481, -0.44280744,
 -1.0597594 ,  0.54360831, -0.6984303 , -0.93138063, -1.05045146,
  0.10548104, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[ 0.87066541,  0.6984303 , -0.26677751, -0.07107726, -0.44280744,
 -1.0597594 , -0.09326954, -0.6984303 ,  0.46569032,  0.55148702,
  1.9438648 ,  1.33143805, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[ 0.4353327 ,  0.6984303 , -0.38301629, -0.40289202, -0.44280744,
  0.99247309, -0.7301474 ,  1.43178211,  0.77615053,  0.55148702,
  1.9438648 , -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[ 0.        ,  0.6984303 , -0.73173261,  1.14557686, -0.44280744,
  0.99247309, -1.36702525,  1.43178211,  1.55230105,  0.55148702,
  1.02467292, -0.75106762, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 1.30599811, -1.43178211,  2.52295306, -0.13744021,  2.25831796,
 -1.0597594 ,  0.71344241,  1.43178211, -0.15523011,  0.55148702,
  1.02467292, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[-0.87066541,  0.6984303 , -1.95223974, -0.82319071,  2.25831796,
 -1.0597594 ,  0.33131569, -0.6984303 , -0.93138063, -1.05045146,
 -0.81371085, -0.75106762,  2.57390754, -0.26490647, -1.05045146,
  1.33143805],
[-0.10883318, -1.43178211, -0.38301629, -0.40289202, -0.44280744,
  0.99247309, -1.40948378, -0.6984303 , -0.93138063, -1.05045146,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.10883318, -1.43178211, -0.38301629, -0.64622284, -0.44280744,
 -0.03364316, -0.77260592,  1.43178211,  0.62092042,  0.55148702,
  0.10548104, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[ 0.        , -1.43178211, -1.42916526, -0.44713399, -0.44280744,
 -1.0597594 ,  0.41623274, -0.6984303 ,  0.31046021,  0.55148702,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
```

```
[-0.97949858,  0.6984303 ,  0.43065513,  1.65435949, -0.44280744,
  0.99247309, -0.05081102,  1.43178211, -0.93138063,  0.55148702,
  1.9438648 , -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[-1.30599811,  0.6984303 , -0.84797138,  0.1280116 ,  2.25831796,
 -1.0597594 ,  1.94473959, -0.6984303 , -0.31046021,  2.1534255 ,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[-1.19716494, -1.43178211, -0.73173261, -0.46925497, -0.44280744,
 -1.0597594 ,  0.71344241, -0.6984303 , -0.77615053,  0.55148702,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.65299906,  0.6984303 , -0.55737445,  0.52618931, -0.44280744,
  0.99247309, -0.30556216,  1.43178211,  1.24184084,  0.55148702,
  0.10548104, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[-0.32649953,  0.6984303 , -2.35907545, -0.1595612 , -0.44280744,
 -1.0597594 ,  0.24639865,  1.43178211, -0.93138063, -1.05045146,
  0.10548104,  1.33143805, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[-1.08833176,  0.6984303 , -0.26677751, -0.02683529, -0.44280744,
 -1.0597594 ,  1.30786174,  1.43178211, -0.62092042, -1.05045146,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.54416588, -1.43178211,  2.29047552,  0.32710045, -0.44280744,
 -1.0597594 , -0.22064511,  1.43178211, -0.93138063,  0.55148702,
 -0.81371085, -0.75106762, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.76183223,  0.6984303 ,  0.31441636, -0.60198087, -0.44280744,
  0.99247309, -0.43293773,  1.43178211,  0.54330537, -1.05045146,
  0.10548104, -0.75106762, -0.38851434, -0.26490647, -1.05045146,
  1.33143805],
[ 0.4353327 ,  0.6984303 , -1.19668771,  1.85344834, -0.44280744,
 -0.03364316, -0.34802069, -0.6984303 ,  2.48368168,  2.1534255 ,
  1.9438648 , -0.75106762, -0.38851434, -0.26490647, -1.05045146,
 -0.75106762],
[ 0.76183223,  0.6984303 , -0.03429997, -0.00471431, -0.44280744,
 -1.0597594 , -0.13572807, -0.6984303 ,  1.08661074,  0.55148702,
  1.02467292, -0.75106762, -0.38851434,  3.77491722,  0.95197164,
 -0.75106762],
[ 0.97949858, -1.43178211,  0.02381942,  0.3934634 , -0.44280744,
  0.99247309,  1.01065207, -0.6984303 , -0.93138063, -1.05045146,
 -0.81371085,  1.33143805, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762],
[ 0.10883318, -1.43178211, -0.15053874,  2.38435196, -0.44280744,
 -1.0597594 ,  0.75590093, -0.6984303 ,  0.        , -1.05045146,
 -0.81371085, -0.75106762,  2.57390754, -0.26490647,  0.95197164,
 -0.75106762],
[ 1.74133082,  0.6984303 , -0.26677751,  1.94193228, -0.44280744,
  0.99247309, -1.66423492, -0.6984303 ,  0.93138063,  0.55148702,
  1.9438648 , -0.75106762, -0.38851434, -0.26490647,  0.95197164,
 -0.75106762]])
```

6.3. Train the Binary Logistic Regression Using Machine Learning Approach

In [70]:

```
## Initialization of the Training Process
ml_model = LogisticRegression(random_state = 42,
```

```
                                max_iter = 1000)

## Training the Binary Logistic Regression Model
ml_model.fit(X_train, y_train)
```

Out[70]:

▾ LogisticRegression ⓘ ?

▸ Parameters

▸ Fitted attributes

6.4. Prediction

In [71]:

```
y_pred = ml_model.predict(X_test)
y_pred
```

Out[71]:

```
array([0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0,
       1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1,
       0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1])
```

In [72]:

```
y_pred_prob = ml_model.predict_proba(X_test)[:,1]
y_pred_prob
```

Out[72]:

```
array([0.22598883, 0.59648954, 0.02755887, 0.02356915, 0.37835796,
       0.01518444, 0.08517744, 0.20041266, 0.80089251, 0.29006779,
       0.89514104, 0.01533138, 0.03835123, 0.96581645, 0.89237746,
       0.96148988, 0.78313808, 0.29306298, 0.83303999, 0.93045106,
       0.65365158, 0.04730726, 0.81456636, 0.52799289, 0.75516574,
       0.03076922, 0.04531406, 0.92527035, 0.20327994, 0.88733325,
       0.25315649, 0.90247458, 0.73264561, 0.41503652, 0.16155691,
       0.91809441, 0.15390543, 0.19083878, 0.04635737, 0.73307932,
       0.93519126, 0.9954347 , 0.97184471, 0.91469083, 0.12094165,
       0.03385886, 0.83744415, 0.01892729, 0.9963088 , 0.27317891,
       0.01818986, 0.96812132, 0.41124844, 0.07734585, 0.3510791 ,
       0.93638813, 0.98899008, 0.43818099, 0.01726862, 0.03379697,
       0.98334472])
```

6.5. Evaluation

In [73]:

```
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"ROC-AUC: {roc_auc_score(y_test, y_pred_prob):.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

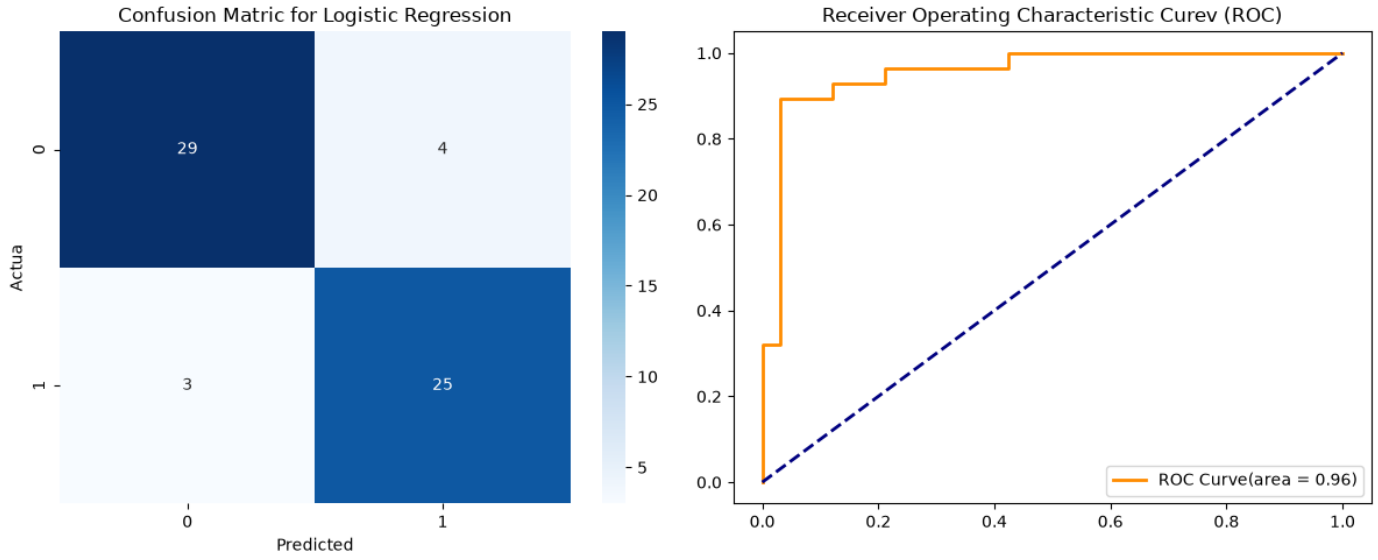
```
Accuracy: 0.8852
ROC-AUC: 0.9556
Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.88      0.89        33
```

```
           1       0.86      0.89      0.88        28

    accuracy                           0.89        61
   macro avg       0.88      0.89      0.88        61
weighted avg       0.89      0.89      0.89        61
```

6.6. Plotting the Confusion Matrix and ROC Curve

In [80]:

```
fig, ax = plt.subplots(1,2, figsize = (12, 5))

## Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot = True, fmt = 'd', cmap = 'Blues', ax = ax[0])
ax[0].set_title('Confusion Matric for Logistic Regression')
ax[0].set_xlabel('Predicted')
ax[0].set_ylabel('Actua')

## ROC-AUC graph
fpr, tpr, threshold = roc_curve(y_test, y_pred_prob)
ax[1].plot(fpr, tpr, color = 'darkorange', lw = 2,
           label = f'ROC Curve(area = {roc_auc_score(y_test, y_pred_prob):.2f})')
ax[1].plot([0,1],[0,1],color = 'navy', lw = 2, linestyle = '--')
ax[1].set_title('Receiver Operating Characteristic Curev (ROC)')
ax[1].legend(loc='lower right')

plt.tight_layout()
plt.show()
```



NEXT SESSION: APPLICATION OF MACHINE AND DEEP LEARNING

In [ ]: